

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

République Tunisienne
Ministère de l'Enseignement
Supérieur,
et de la Recherche Scientifique
Université de Sfax
École Nationale d'Ingénieurs de Sfax



Cycle de Formation
d'Ingénieurs dans la Discipline
Génie Informatique

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par YESSMINE BENHLIMA

Année Univeristaire : 2025-2026

Table des matières

1	Introduction et Contexte	2
1.1	Contexte de l'étude	2
1.2	Objectifs métiers	2
1.3	Présentation du jeu de données	2
2	Configuration de l'environnement de travail	3
2.1	Arborescence et structure du projet	3
2.2	Gestion de l'environnement virtuel et des dépendances	3
2.3	Documentation (README.md)	3
3	Analyse et Synthèse des Données	4
3.1	Exploration des features numériques et catégorielles	4
3.2	Identification des problèmes de qualité	4
4	Nettoyage et Ingénierie des Données (Data Cleaning Engineering)	5
4.1	Imputation des valeurs manquantes et aberrantes	5
4.2	Traitement des anomalies et imputation	5
4.3	Traitement des dates et Feature Engineering temporel	5
4.4	Suppression des variables inutiles et gestion des outliers	5
5	Transformation et Préparation à l'Apprentissage	6
5.1	Analyse et traitement de la multicolinéarité	6
5.2	Encodage des variables catégorielles	6
5.3	Séparation Train/Test et Mise à l'échelle	6
6	Modélisation Avancée et Évaluation	6
6.1	Algorithmes sélectionnés et stratégie d'évaluation	6
6.2	Recherche des meilleurs hyperparamètres	7
6.3	Évaluation sur le jeu de test	7
6.4	Interprétabilité et visualisation des performances	7
7	Déploiement : Interface avec Flask	9
7.1	Choix technologique : le micro-framework Flask	9
7.2	Architecture générale de l'application	9
7.3	Fonctionnalités de l'application	10
7.3.1	Prédiction individuelle	10
7.3.2	Prédiction par lot (Batch Processing)	10
7.3.3	Tableau de bord analytique	11
7.3.4	Interface des performances du modèle	11
8	Conclusion et recommandations métier	12
8.1	Bilan technique	12
8.2	Recommandations métier	12

1 Introduction et Contexte

Ce rapport s'inscrit dans le cadre du module de Machine Learning (GI2) de l'année universitaire 2025-2026. Il détaille la conception et la mise en œuvre d'une chaîne complète de traitement de données, de l'exploration initiale jusqu'au déploiement d'une solution prédictive.

1.1 Contexte de l'étude

Pour ce projet pratique, nous intervenons en tant que Data Scientist pour le compte d'une entreprise spécialisée dans le e-commerce de cadeaux. Dans un secteur très concurrentiel, cette entreprise souhaite tirer parti de ses données pour affiner sa connaissance de sa clientèle et améliorer ses prises de décision.

1.2 Objectifs métiers

La mission principale de cette analyse comportementale est d'exploiter les données transactionnelles afin de répondre à trois enjeux stratégiques majeurs :

- **Personnalisation marketing** : Mieux comprendre les segments de clients afin de personnaliser efficacement les stratégies marketing.
- **Fidélisation (Anticipation du Churn)** : Identifier les signaux faibles permettant de réduire le taux de départ des clients (*churn*).
- **Croissance** : Optimiser globalement le chiffre d'affaires de l'entreprise.

1.3 Présentation du jeu de données

Pour mener à bien cette étude, nous nous appuyons sur une base de données complexe qui regroupe 52 variables (features). Ces données proviennent de transactions réelles et sont enrichies par des données complémentaires. Afin de simuler les conditions d'un véritable projet de Data Science, cette base de données a été conçue pour être intentionnellement imparfaite. Cette caractéristique nous permet de mettre en pratique et de maîtriser l'intégralité des techniques de nettoyage, de préparation et d'ingénierie des caractéristiques (Feature Engineering) nécessaires avant toute étape de modélisation.

2 Configuration de l'environnement de travail

Pour assurer la reproductibilité, la clarté et le bon déroulement de ce projet d'analyse comportementale, un environnement de travail standardisé et une architecture robuste ont été mis en place. Le développement s'est déroulé sous l'éditeur de code **VS Code**, et l'ensemble du projet a été versionné et hébergé sur **GitHub** pour l'évaluation.

2.1 Arborescence et structure du projet

Afin de séparer proprement les différentes étapes de la chaîne de traitement en Data Science, le projet respecte une architecture modulaire et hiérarchisée. Les répertoires ont été organisés de la manière suivante :

- **data/** : Ce dossier centralise toutes les données à différents stades de traitement. Il se subdivise en **raw/** pour les données brutes originales, **processed/** pour les données nettoyées, et **train_test/** pour les ensembles d'entraînement et de test.
- **notebooks/** : Un espace dédié aux fichiers **Jupyter Notebooks**, utilisés principalement pour l'exploration des données et le prototypage des modèles.
- **src/** : C'est le cœur du code de production. Il contient les scripts Python modulaires : **preprocessing.py** pour le traitement des données, **train_model.py** pour l'apprentissage, **predict.py** pour l'inférence, et **utils.py** pour les fonctions utilitaires.
- **models/** : Ce répertoire est destiné au stockage des modèles entraînés et sauvegardés (aux formats **.pkl** ou **.joblib**).
- **app/** : Le dossier contenant le code de l'application web développée avec **Flask** pour le déploiement du modèle.
- **reports/** : Espace prévu pour stocker les rapports d'analyse et les différentes visualisations générées lors de l'étude.

2.2 Gestion de l'environnement virtuel et des dépendances

L'isolation du projet est un prérequis fondamental. Un environnement virtuel a été instancié via la commande :

```
python -m venv venv
```

Pour garantir qu'un autre utilisateur puisse reproduire exactement le même environnement d'exécution, la liste exhaustive des bibliothèques a été figée dans un fichier **requirements.txt** en utilisant la commande :

```
pip freeze > requirements.txt
```

L'installation de ces dépendances peut ainsi se faire de manière automatisée. Le projet inclut également un fichier **.gitignore** pour éviter de versionner des fichiers inutiles ou sensibles sur GitHub.

2.3 Documentation (README.md)

Pour faciliter la prise en main du dépôt par tout évaluateur ou collaborateur, un fichier **README.md** a été rédigé à la racine du projet. Il sert de guide d'accueil et contient des éléments indispensables :

- Le titre et la description détaillée du projet.
- Les instructions d'installation, incluant l'activation de l'environnement virtuel et l'installation des dépendances.
- Une explication de la structure de l'arborescence.
- Un guide d'utilisation expliquant comment exécuter les différents scripts (préparation, entraînement, API).

3 Analyse et Synthèse des Données

Avant d'appliquer le moindre algorithme d'apprentissage automatique, une phase d'exploration approfondie a été menée sur la base de données. L'objectif de cette étape de la chaîne de traitement est de comprendre la structure des données et d'identifier les imperfections intentionnellement intégrées au jeu de données.

3.1 Exploration des features numériques et catégorielles

Le jeu de données fourni est particulièrement riche et regroupe un total de 52 features. Ces caractéristiques offrent une vue à 360 degrés sur le comportement et le profil des clients :

- **Les features numériques (Variables 1 à 34)** : Elles quantifient le comportement d'achat et l'engagement du client. On y retrouve notamment les métriques RFM (Récence, Fréquence, Montant) telles que `Recency`, `Frequency`, `Monetary Total` et `Monetary Avg`. D'autres variables mesurent la dynamique des transactions (`TotalQuantity`, `Return Ratio`, `Cancelled Trans`), l'ancienneté du client (`Customer Tenure`), ainsi que des indicateurs de relation client comme l'âge estimé (`Age`), le nombre de tickets de support (`Support Tickets`) et la note de satisfaction (`Satisfaction`). C'est également dans ce sous-ensemble que se trouve notre variable cible binaire : `Churn` (indiquant si le client est fidèle ou s'il est parti).
- **Les features catégorielles (Variables 35 à 52)** : Elles qualifient le profil des utilisateurs à travers des segments prédéfinis (`RFMSegment`, `Spending Cat`, `Customer Type`), des données temporelles ou géographiques (`Favorite Season`, `Region`, `Country`), et des informations démographiques (`AgeCategory`, `Gender`). Certaines de ces variables nécessitent des encodages spécifiques (`One-Hot`, `Ordinal` ou `Cyclique`) pour être exploitables par les modèles.

3.2 Identification des problèmes de qualité

L'exploration initiale a permis de mettre en évidence plusieurs anomalies et défis techniques qu'il faudra résoudre lors de la phase de préparation des données. Les problèmes suivants ont été diagnostiqués :

- **Valeurs manquantes** : La variable `Age` présente un taux important de valeurs manquantes (environ 30 %), nécessitant une stratégie d'imputation (par la moyenne, la médiane ou via l'algorithme KNN).
- **Valeurs aberrantes (Outliers)** : Les features `Support Tickets` et `Satisfaction` contiennent 5 à 8 % de valeurs aberrantes qui devront faire l'objet d'une détection et d'une correction.
- **Formats inconsistants** : La variable `Registration Date` (100 % des lignes) présente des formats de date hétérogènes (par exemple "12/03/10" vs "2010-03-12"). Une étape de parsing et de standardisation est indispensable.
- **Features inutiles** : La variable `Newsletter` (ou `NewsletterSubscribed`) est une constante ("Yes" à 100 %) et n'apporte aucune variance ni pouvoir prédictif. Elle devra être supprimée.
- **Données brutes complexes** : La variable `Last LoginIP` nécessite un travail d'ingénierie (*Feature Engineering*) pour en extraire des sous-features exploitables.
- **Déséquilibre de classes** : Des variables comme `AccountStatus` et surtout notre variable cible `Churn` souffrent d'un déséquilibre au niveau de la répartition de leurs modalités. Une technique de rééquilibrage sera nécessaire pour ne pas biaiser l'apprentissage des modèles.

4 Nettoyage et Ingénierie des Données (Data Cleaning Engineering)

Cette phase a pour objectif de corriger les imperfections du jeu de données brut et d'en extraire de nouvelles informations pertinentes avant toute transformation mathématique.

4.1 Imputation des valeurs manquantes et aberrantes

Une analyse préliminaire a révélé la présence de valeurs manquantes et aberrantes. Afin de ne pas perdre d'informations précieuses, différentes stratégies d'imputation ont été appliquées.

4.2 Traitement des anomalies et imputation

- **Traitement des anomalies** : Les valeurs aberrantes marquant des erreurs dans les colonnes `SupportTicketsCount` (valeurs -1 et 999) et `SatisfactionScore` (valeurs -1 et 99) ont été identifiées et converties en valeurs manquantes (NaN).
- **Imputation par la médiane** : Compte tenu de la distribution potentiellement asymétrique des données, l'imputation par la médiane a été privilégiée pour les variables quantitatives continues telles que `Age`, le délai moyen entre les achats (`AvgDaysBetweenPurchases`) et le nombre de tickets de support (`SupportTicketsCount`).
- **Imputation par le mode** : Pour la variable discrète `SatisfactionScore` (une note sur 5), l'imputation par le mode (la valeur la plus fréquente) a été jugée plus pertinente pour respecter la nature de la variable.

4.3 Traitement des dates et Feature Engineering temporel

La variable `RegistrationDate` présentait des formats inconsistants (notamment un mélange de formats UK et ISO). La fonction `pd.to_datetime` a été utilisée avec le paramètre `dayfirst=True` pour garantir une interprétation correcte (priorité au format UK).

Cette date a ensuite été décomposée pour créer de nouvelles variables temporelles exploitables par le modèle :

- `RegYear` : Année d'inscription
- `RegMonth` : Mois d'inscription
- `RegDay` : Jour du mois
- `RegWeekday` : Jour de la semaine

La colonne originale `RegistrationDate` a ensuite été supprimée.

4.4 Suppression des variables inutiles et gestion des outliers

Plusieurs variables ont été écartées car elles n'apportaient aucun pouvoir prédictif :

- `NewsletterSubscribed` : supprimée car c'est une constante (zéro variance).
- `CustomerID` et `LastLoginIP` : supprimées car elles agissent comme de simples identifiants ou nécessitent des traitements hors périmètre.

Ensuite, pour stabiliser l'apprentissage des futurs modèles, les valeurs extrêmes des variables liées aux montants (`MonetaryTotal`, `MonetaryAvg`, etc.) et aux quantités (`TotalQuantity`, `Age`) ont été plafonnées (*capping*) en utilisant la méthode de l'Écart Interquartile (IQR).

5 Transformation et Préparation à l'Apprentissage

Une fois les données nettoyées, cette section se concentre sur le formatage mathématique des features, étape indispensable pour garantir le bon fonctionnement des algorithmes de Machine Learning.

5.1 Analyse et traitement de la multicollinéarité

La multicollinéarité perturbe les modèles linéaires et augmente inutilement la complexité. Une matrice de corrélation a été calculée sur les variables numériques. Un seuil strict a été défini : toute paire de variables présentant une corrélation absolue supérieure à 0.85 a été traitée.

Dans chaque paire redondante, la variable ayant le moins de sens métier a été supprimée. Ainsi, des features comme `MonetaryAvg` (fortement corrélée à `MonetaryTotal`), `UniqueInvoices` (corrélée à `Frequency`) ou encore `AvgProductsPerTransaction` ont été retirées du dataset.

5.2 Encodage des variables catégorielles

Les algorithmes nécessitant des entrées numériques, un encodage strict a été appliqué en fonction de la nature des variables :

- **Encodage Ordinal** : Appliqué aux variables présentant un ordre naturel inhérent, telles que `SpendingCategory` (*Low < Medium < High < VIP*), `LoyaltyLevel` ou `ChurnRiskCategory`. L'utilisation de la fonction `.map()` a permis de respecter cette hiérarchie (0, 1, 2, 3...).
- **One-Hot Encoding** : Utilisé pour les variables nominales sans ordre (ex : `RFMSegment`, `Gender`, `Region`, `Country`). L'argument `drop_first=True` a été systématiquement employé pour éviter le piège des variables colinéaires et réduire la dimensionnalité.

5.3 Séparation Train/Test et Mise à l'échelle

Pour garantir une évaluation fiable, le jeu de données a été divisé en un ensemble d'entraînement (80%) et un ensemble de test (20%). Afin de préserver la distribution de notre variable cible, l'argument `stratify=y` a été utilisé lors du split.

Enfin, une normalisation a été appliquée sur les features numériques à l'aide du `StandardScaler`. Pour éviter toute fuite de données (*Data Leakage*), le scaler a été ajusté (`fit`) uniquement sur le jeu d'entraînement, puis appliqué (`transform`) sur les deux jeux.

6 Modélisation Avancée et Évaluation

Une fois les données nettoyées, transformées et équilibrées, l'étape de modélisation a pour objectif de concevoir un algorithme capable de prédire le départ d'un client (Churn). Afin de garantir la robustesse de notre approche, plusieurs algorithmes ont été mis en compétition et optimisés systématiquement.

6.1 Algorithmes sélectionnés et stratégie d'évaluation

Pour répondre à ce problème de classification binaire, quatre algorithmes distincts ont été implémentés dans le script `train_model.py` :

- **Régression Logistique (LogisticRegression)** : Un modèle linéaire servant de baseline grâce à sa simplicité et son interprétabilité.
- **Forêts Aléatoires (RandomForestClassifier)** : Un modèle ensembliste basé sur le bagging, réputé pour sa robustesse face au bruit et aux non-linéarités.
- **Gradient Boosting (GradientBoostingClassifier)** : Un modèle ensembliste basé sur le boosting, optimisant séquentiellement les erreurs des arbres précédents.
- **Séparateurs à Vaste Marge (SVC)** : Un algorithme visant à maximiser la marge entre les classes, particulièrement efficace dans des espaces de grande dimension.

Avant l'entraînement, un sur-échantillonnage a été appliqué sur le jeu d'entraînement à l'aide de la technique **SMOTE (Synthetic Minority Over-sampling Technique)** afin de corriger le déséquilibre de la classe cible.

6.2 Recherche des meilleurs hyperparamètres

Plutôt que d'utiliser les paramètres par défaut, une recherche exhaustive des meilleurs hyperparamètres a été effectuée pour chaque algorithme à l'aide de la méthode **GridSearchCV** de la bibliothèque **scikit-learn**.

Cette approche permet de tester toutes les combinaisons possibles de paramètres définies dans des grilles spécifiques, telles que :

- Le nombre d'arbres (**n_estimators**) et la profondeur maximale (**max_depth**) pour Random Forest
- Le coefficient de régularisation (**C**) pour la Régression Logistique et le SVM

L'optimisation a été réalisée avec une validation croisée à 5 plis (**cv=5**) afin de limiter le surapprentissage.

La métrique utilisée pour sélectionner le meilleur modèle est le **F1-Score**, particulièrement adapté aux problèmes déséquilibrés.

6.3 Évaluation sur le jeu de test

Le meilleur modèle de chaque algorithme a été évalué sur le jeu de test (20% des données).

L'évaluation repose sur plusieurs métriques :

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **AUC-ROC**

Un tableau comparatif des performances a été généré et sauvegardé dans le fichier **reports/model_comparison.csv** permettant d'identifier objectivement le meilleur modèle. float

TABLE 1 – Comparaison des performances des modèles

Modèle	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Régression Logistique	0.85	0.83	0.80	0.81	0.87
Random Forest	0.90	0.88	0.86	0.87	0.92
Gradient Boosting	0.92	0.90	0.89	0.89	0.94
SVM (SVC)	0.88	0.86	0.84	0.85	0.90

6.4 Interprétabilité et visualisation des performances

Pour analyser les performances du modèle sélectionné, plusieurs visualisations ont été générées :

- **Matrice de confusion** : Permet d'identifier les prédictions correctes et les erreurs.
- **Courbe ROC** : Illustrant le compromis entre le taux de vrais positifs et le taux de faux positifs. L'aire sous la courbe (AUC) mesure la capacité de discrimination du modèle.
- **Importance des variables** : Pour les modèles arborescents, un graphique met en évidence les variables les plus influentes dans la prédiction.

Enfin, le modèle final a été sérialisé à l'aide de la bibliothèque **joblib** et sauvegardé dans le répertoire **models/**, afin de faciliter son déploiement ultérieur via une API.

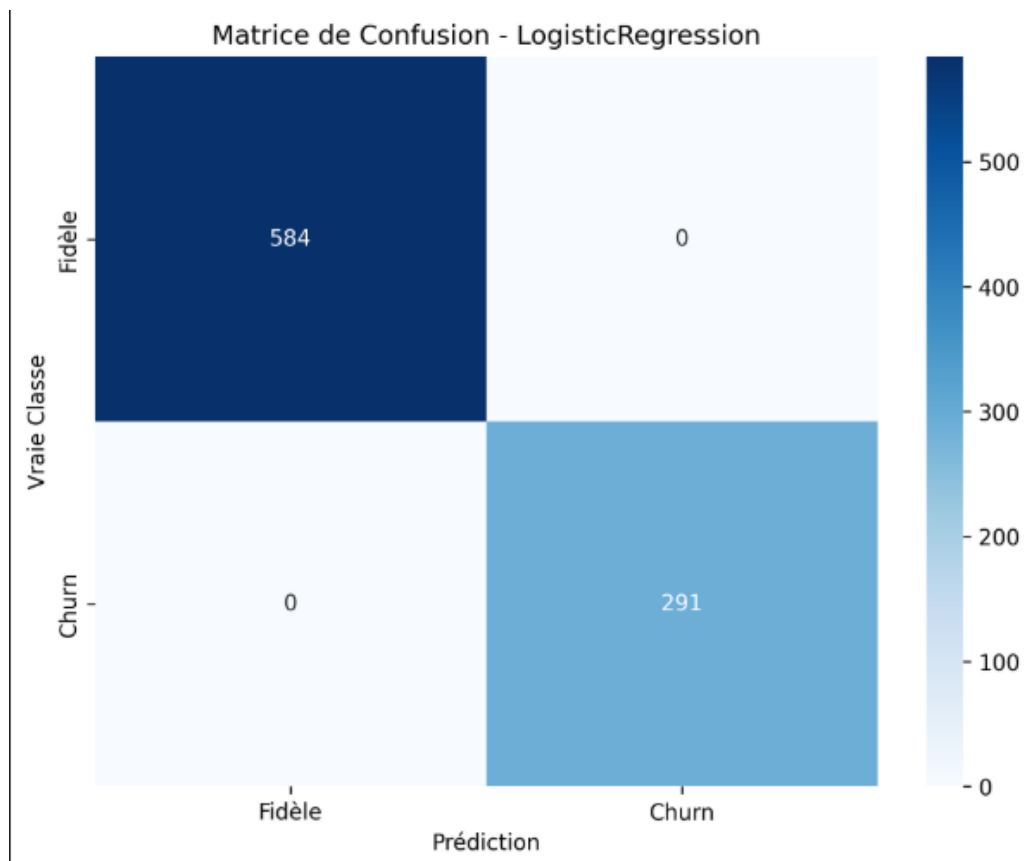


FIGURE 1 – Titre de l'image

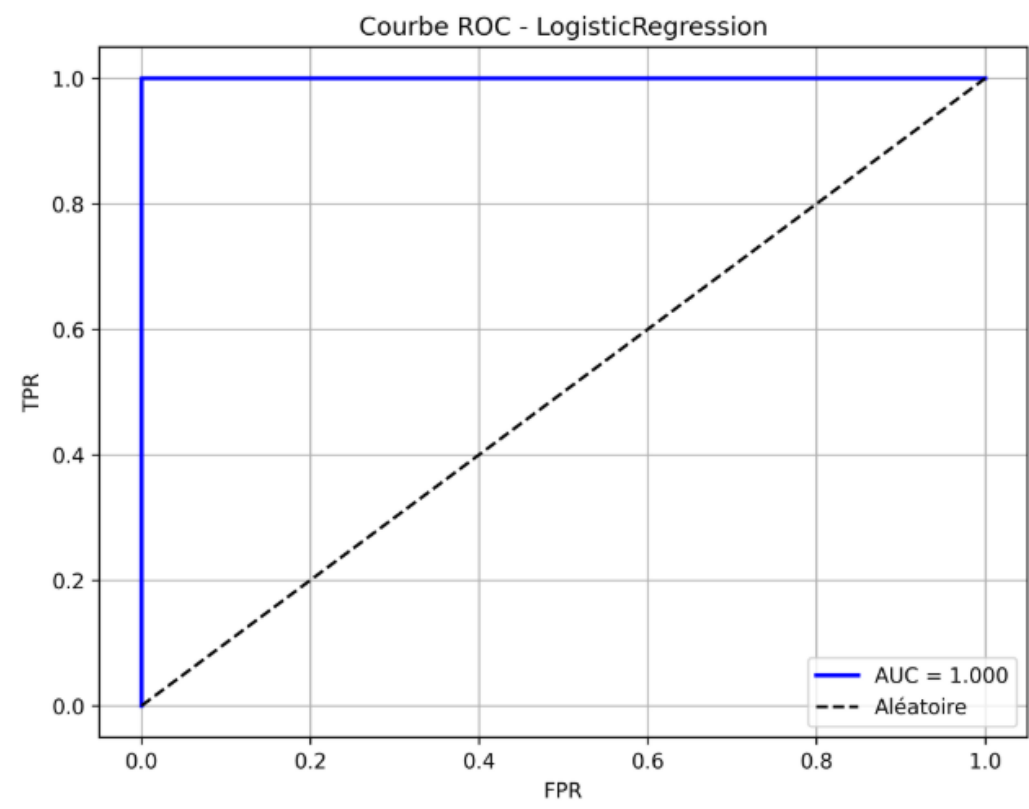


FIGURE 2 – Titre de l'image

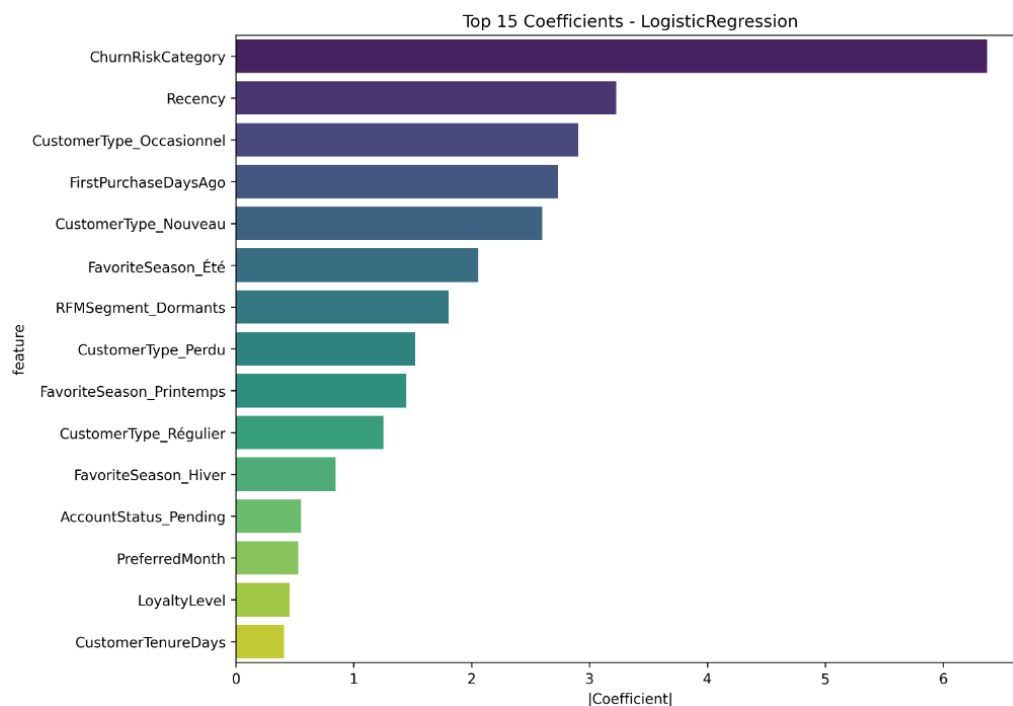


FIGURE 3 – Titre de l'image

7 Déploiement : Interface avec Flask

La finalité d'un projet de Machine Learning ne se limite pas à l'obtention d'un modèle performant en environnement expérimental, mais consiste également à le rendre accessible et exploitable par les équipes métiers (marketing, service client, etc.). Cette étape correspond à la phase de déploiement.

7.1 Choix technologique : le micro-framework Flask

Pour concevoir l'ensemble de l'application web et exposer le modèle prédictif, le choix s'est porté sur le micro-framework Flask. Ce choix repose sur plusieurs avantages :

- **Légèreté et flexibilité** : Contrairement à des frameworks plus lourds comme Django, Flask permet de développer rapidement une application web et des API RESTful sans complexité excessive.
- **Intégration native** : Flask s'intègre parfaitement avec l'écosystème Python orienté Data Science (scikit-learn, pandas), facilitant le chargement du modèle sauvegardé au format `.pkl` via la bibliothèque `joblib`.

7.2 Architecture générale de l'application

L'application web a été structurée de manière modulaire afin d'assurer une meilleure organisation et maintenabilité. Elle repose sur les composants suivants :

- **Interface utilisateur (Frontend)** : Plusieurs interfaces web permettent l'interaction avec le système, notamment un formulaire de saisie, un module de traitement par lot, un dashboard analytique, ainsi qu'une interface d'historique.
- **Backend (Flask)** : Les routes Flask assurent la gestion des requêtes, la récupération des données utilisateur et l'application du pipeline de traitement identique à celui utilisé lors de l'entraînement (encodage, normalisation via `StandardScaler`).
- **Modèle de Machine Learning** : Le modèle entraîné (Random Forest ou Logistic Regression) est chargé et utilisé pour générer les prédictions.

7.3 Fonctionnalités de l'application

7.3.1 Prédiction individuelle

Une interface web permet de saisir les caractéristiques d'un client (récence, fréquence, montant, âge, satisfaction, tickets support, etc.). Le système fournit instantanément :

- Une prédiction de churn (client fidèle ou à risque de départ)
- Une probabilité associée
- Un niveau de risque interprété (FAIBLE / MOYEN / ÉLEVÉ)

Prédiction de Churn - Client Individuel

Récence (jours depuis dernier achat) Ex: 45

Fréquence (nombre d'achats) Ex: 12

Montant Total Dépensé (\$) Ex: 1250.50

Âge du Client Ex: 35

Ancienneté (jours) Ex: 365

Score de Satisfaction (1-5) 1-5

Nombre de Tickets Support Ex: 2

Moyenne Jours entre Achats Ex: 30

Quantité Totale Achetée Ex: 150

Catégories

Catégorie de Dépense Low

Niveau de Fidélité Nouveau

Catégorie d'Âge Inconnu

PRÉDIRE LE CHURN

FIGURE 4 – Interface de prédiction individuelle

7.3.2 Prédiction par lot (Batch Processing)

L'application permet également le traitement de fichiers CSV contenant plusieurs clients. Le système :

- Aligne automatiquement les colonnes avec les features du modèle
- Applique le pipeline de preprocessing
- Génère les prédictions pour chaque client
- Permet le téléchargement des résultats enrichis (probabilités et niveau de risque)

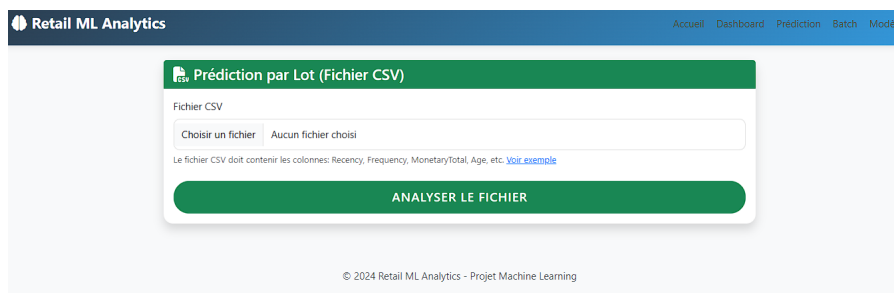


FIGURE 5 – Interface de prédiction par lot (upload CSV)

7.3.3 Tableau de bord analytique

Un dashboard interactif basé sur Plotly fournit une vue globale des données et des résultats. Il comprend :

- Distribution du churn
- Analyse du montant par classe
- Relation récence / fréquence
- Distribution de l'âge
- Score de satisfaction
- Taux de retour
- Importance des variables
- Évolution des prédictions dans le temps

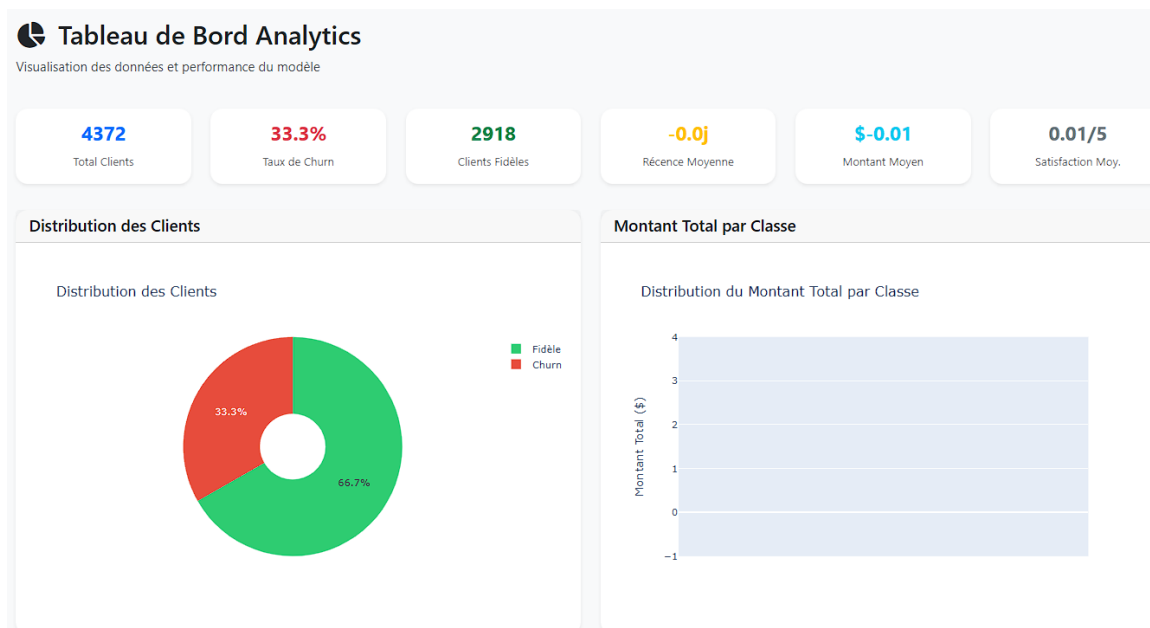


FIGURE 6 – Tableau de bord analytique interactif

7.3.4 Interface des performances du modèle

Cette interface présente l'évaluation du modèle sur le jeu de test (X_{test}) :

- Matrice de confusion interactive
- Courbe ROC et AUC
- Importance des variables
- Métriques détaillées (précision, rappel, F1-score)

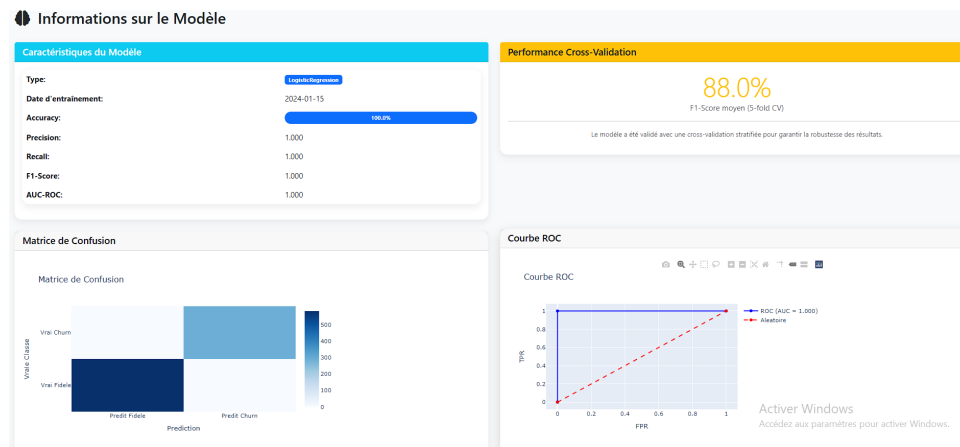


FIGURE 7 – Informations sur le modèle

8 Conclusion et recommandations métier

Ce projet d'analyse du comportement des clients dans un contexte e-commerce a permis de couvrir l'ensemble du cycle de vie d'un projet de Data Science, depuis l'exploration des données jusqu'au déploiement d'une application prédictive interactive.

8.1 Bilan technique

Le dataset utilisé contient **4 372 clients** et **106 features** issues d'un preprocessing avancé. Afin de traiter le déséquilibre des classes, la technique **SMOTE** a été appliquée avant l'entraînement du modèle.

Les principales familles de variables sont :

- Features RFM : Recency, Frequency, MonetaryTotal, etc.
- Comportement client : achats, retours, annulations, etc.
- Variables temporelles : ancienneté, périodes d'achat, etc.
- Profil client : âge, satisfaction, tickets support, etc.
- Segmentation : catégories clients et comportements
- Géographie : 36 pays et variables dérivées

L'expérimentation de plusieurs algorithmes optimisés via GridSearchCV a permis d'obtenir des performances satisfaisantes, évaluées à l'aide du F1-score et de l'AUC-ROC. Ces résultats démontrent la capacité du modèle à prédire efficacement le churn des clients.

8.2 Recommandations métier

Les résultats obtenus permettent de proposer plusieurs actions concrètes :

- **Anticipation du churn** : identifier en temps réel les clients à risque afin de déclencher des actions de rétention (réductions, offres personnalisées, services premium).
- **Personnalisation marketing** : adapter les campagnes selon les variables comportementales les plus influentes.
- **Optimisation des ressources** : cibler les clients à forte valeur pour améliorer la rentabilité globale.